GTSparse: A Geometric-Template-Driven Sparse Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Sparse convolution is the dominant computational primitive in 3D point-cloud perception. Its execution challenge lies in that each output voxel's set of active offsets (those pointing to non-empty input neighbors) is irregular and determined only at runtime. Existing work sit at opposite ends of a fundamental tension: some pursue no wasted computation by gathering each offset's active output-offset pairs into an independent dense multiplication, but this fragments each output's computation across many separate operations, sacrificing throughput; others maximize throughput by fusing all offsets of each output voxel into a single unified dense matrix multiplication, but inevitably waste arithmetic on inactive offsets.

We present GTSparse, a sparse convolution runtime that resolves this tension through *geometric templates*. We observe that the active-neighbor patterns of output voxels, although individually irregular, concentrate on a small number of recurring structures. GTSparse defines a compact set of geometric templates (fixed offset subsets), assigns each output voxel exclusively to its narrowest compatible template, and executes each template group as a narrower implicit GEMM. Within each template, computation is fully structured; across templates, narrower widths eliminate wasted work on empty offsets. On representative detection and segmentation workloads across multiple GPUs, GTSparse achieves **XX–XX**× end-to-end speedup over state-of-the-art sparse convolution engines, while producing numerically identical results.

1 Introduction

Sparse convolution has become a key computational primitive in 3D point cloud perception, playing a central role in autonomous driving [9, 11, 14], robotics [12, 13, 20], and AR/VR [5, 8, 18]. Its importance stems from the intrinsic spatial sparsity of these domains: input features reside on 3D voxel grids in which only a tiny fraction of voxels are active. In widely used benchmarks, the active ratio (the fraction of occupied voxels) is extremely low: around 0.01% on KITTI [7], about 0.05% on nuScenes [2], and roughly 1% on SemanticKITTI [1]. Directly applying dense convolution would therefore waste substantial computation and memory on empty regions. The central challenge is thus not the convolution operation itself, but how to efficiently organize and execute the small number of irregularly distributed, input-dependent computations on GPUs.

Consider a 3×3×3 sparse convolution kernel: each output voxel is associated with 27 spatial offsets, each pointing to a potential input neighbor. Under low-density inputs, most of these neighbors are empty; computation is needed only where a neighbor actually exists. We call such an (output, offset) pair an *active pair*. Unlike sparse matrix multiplication where the nonzero pattern is static, the set of active pairs here is determined entirely by the runtime distribution of input voxels and changes every frame. Each offset carries its own dense weight matrix shared across all its active pairs, so the operator repeatedly applies offset-specific weights over a dynamically generated, irregular set of spatial pairs. The number and distribution of active pairs cannot be known ahead of time, making efficient GPU scheduling fundamentally difficult.

Existing GPU sparse convolution engines sit at opposite ends of a fundamental tension. On one end is *exact work*: the total computation is proportional to the number of genuinely active pairs, with no arithmetic spent on empty neighbors. Per-offset engines such as MinkowskiEngine [4] achieve this by collecting active pairs separately for each offset and executing an independent multiplication per offset. The consequence is that a single output voxel's full result is scattered across up to 27 separate operations that must be combined externally, preventing a cohesive, GPU-friendly execution.

On the other end is *structured computation*: all contributions to a single output are produced within one unified dense matrix multiplication for high throughput. Implicit-GEMM engines such as SpConv v2 [6] and TorchSparse++ [17] achieve this by fusing all 27 offsets into a single GEMM. The consequence is that every offset position is computed uniformly, so a large fraction of arithmetic is spent on empty neighbors. Per-offset engines achieve exactness without structure (achieved only XX% throughput as the implicit-GEMM engines in our evaluation); implicit-GEMM engines achieve structure without exactness (wasted XX% of its computation on empty neighbors in our evaluation). Neither converts the GPU's raw capability into effective throughput on genuinely active pairs.

Our key observation is that the active-neighbor patterns of output voxels, although individually irregular, concentrate on a small number of recurring structures, making it unnecessary to choose between fusing all 27 offsets and treating each independently. This concentration arises because active voxels in real scenes follow physical regularities (surfaces, edges, clusters) that limit which offset combinations actually co-occur. We can therefore define a small set of *geometric templates*, each a fixed subset of offsets, and assign every

output voxel exclusively to the narrowest template that covers all its active neighbors. Output voxels sharing the same template then execute together as a single dense matrix multiplication over only that template's offsets. Within each template, computation is fully structured; across templates, narrower widths than the full 27 offsets mean less work is wasted on empty neighbors, approaching exact work.

Realizing this idea requires addressing two challenges. First, *template set design*: the template set must be compact (few templates, to limit execution branches) yet achieve tight coverage (low average assigned width across output voxels). Second, *heterogeneous execution*: output voxels assigned to different templates follow different-width compute paths on the same GPU; scheduling this intra-template homogeneous, inter-template heterogeneous workload efficiently is nontrivial.

We propose GTSparse, a sparse convolution runtime driven by a geometric template set. At a high level, GTSparse constructs a compact template set offline from the kernel's geometric structure, then at runtime assigns each output voxel to its tightest-fitting template, groups voxels by template, and executes each group as a narrower implicit GEMM over only the offsets specified by that template.

For template set design, GTSparse exploits the periodic structure of the kernel offsets to define candidate template families, then uses data-driven greedy selection to assemble a compact set. The resulting templates are fixed and input-independent, constructed once offline and reused across all inputs without modification or runtime cost. On representative workloads, approximately 70% of output voxels are covered by narrower templates, reducing the average number of offsets processed per voxel by XX%–XX% compared to full-width fusion. A full-width fallback template handles the remaining voxels whose patterns do not fit any narrower template.

For heterogeneous execution, GTSparse arranges all output voxels into a single contiguous stream, segmented by template and padded so that each tile (the smallest batch of output voxels processed together by GPU) contains only voxels of the same template. A single kernel launch processes the entire stream: each tile executes a dense matrix multiplication over only its template's offsets, with no cross-template coordination needed. The result is intra-tile homogeneous structured computation at varying widths across tiles, achieved without multiple dispatches or explicit inter-template scheduling. Template assignment is a lightweight runtime step whose overhead we measure and report separately.

We evaluate GTSparse in both FP16 and FP32 precision on three representative model/dataset combinations that span detection and segmentation tasks: SECOND [19] on KITTI [7], VoxelNeXt [3] on nuScenes [2], and SparseResUNet42 [4] on SemanticKITTI [1]. GTSparse achieves **XX–XX**× end-to-end inference speedup over TorchSparse++ [17], **XX–XX**× over SpConv v2 [6], and **XX–XX**× over MinkowskiEngine [4], while producing numerically identical results.

The main contribution of this paper is the geometric template set as an execution abstraction for sparse convolution that resolves the tension between exact work and structured computation. We present a template set construction method that combines a geometry-derived candidate space with data-driven greedy selection, producing a fixed, input-independent, and interpretable template set. On top of this abstraction, we build a geometric-template-driven execution runtime that schedules intra-template homogeneous and inter-template heterogeneous computation within a single GPU kernel. We systematically evaluate GTSparse on real point-cloud models and datasets, demonstrating consistent speedups. By reducing sparse convolution latency, GTSparse enables tighter perception-to-action loops in latency-critical applications such as autonomous driving, where faster 3D inference directly translates to longer decision horizons and improved safety margins.

2 Background and Motivation

This section formalizes the sparse convolution operator (§2.1), surveys the two dominant execution strategies and their limitations (§2.2), positions GTSparse relative to orthogonal optimization directions (§2.3), and quantifies the gap that motivates our design (§2.4).

2.1 Sparse Convolution Primer

Working scenario. 3D point-cloud perception models for autonomous driving [9, 11, 14], robotics [12, 13, 20], and AR/VR [5, 8, 18] typically voxelize raw sensor data (e.g., a LiDAR scan) onto a 3D grid and process it through a backbone composed of multiple sparse convolution layers. Sparse convolution dominates the inference latency of these models. Because each input frame produces a different set of occupied voxels, the computation graph of every sparse convolution layer is input-dependent and must be resolved at runtime.

Computation model. Let $\mathcal{A}_{\text{in}}$ and $\mathcal{A}_{\text{out}}$ denote the sets of active input and output voxels on the discretized 3D grid. A sparse convolution with a $k \times k \times k$ kernel defines $K = k^3$ spatial offsets $\mathcal{D} = \{d_1, \ldots, d_K\}$. The output active set is determined by the input active set and the kernel geometry:

$$\mathcal{A}_{\text{out}} = \left\{ p \mid \exists\, d \in \mathcal{D},\ p + d \in \mathcal{A}_{\text{in}} \right\}. \tag{1}$$

For each output voxel $p \in \mathcal{A}_{\text{out}}$, let $\mathcal{N}(p) = \{d \in \mathcal{D} \mid p+d \in \mathcal{A}_{\text{in}}\}$ denote its set of valid offsets. The sparse convolution computes

$$\mathbf{y}[p] = \sum_{d \in \mathcal{N}(p)} \mathbf{W}[d] \cdot \mathbf{x}[p+d], \tag{2}$$

where $\mathbf{W}[d] \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ is the weight matrix associated with offset d, $\mathbf{x}[p+d] \in \mathbb{R}^{C_{\text{in}}}$ is the input feature at voxel $p+d$, and $\mathbf{y}[p] \in \mathbb{R}^{C_{\text{out}}}$ is the output feature at voxel p.

Submanifold vs. regular sparse convolution. Submanifold (SubM) sparse convolution constrains $\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$, preserving the spatial sparsity pattern across layers. Consequently, for consecutive SubM layers with the same kernel size, the per-offset mapping from each output voxel to its valid input neighbors remains identical and need only be constructed once. In representative 3D detection backbones such as SECOND [19] and VoxelNeXt [3], SubM layers account for the overwhelming majority of convolution layers. All existing engines [4, 6, 17], including GTSparse, exploit this invariance by reusing runtime metadata across SubM layers. Regular (full) sparse convolution applies stride-2 downsampling, producing a new output active set each layer. Its runtime metadata must be rebuilt per layer.

2.2 Existing Execution Strategies

Per-offset engines. Per-offset engines such as MinkowskiEngine [4] partition computation by kernel offset. For each offset d, the engine gathers active pairs into a contiguous buffer, invokes a dense GEMM, and scatters results back to the output. This performs exact work: offsets with no active pairs are skipped entirely. However, the number of active pairs varies widely across offsets, fragmenting the workload into up to K unevenly sized tasks. At low active ratios, many offsets produce only a handful of pairs, yielding thin GEMMs that cannot saturate the GPU. A single output voxel's result is scattered across up to K separate operations that must be combined externally, preventing a cohesive execution for each output. Subsequent systems (TorchSparse [15] adaptive grouping, SpConv v1 fused dispatch) mitigate launch overhead, but per-offset independent execution remains the root cause of low raw throughput.

Implicit-GEMM engines. Implicit-GEMM engines fuse all K offsets into a single dense matrix multiplication, so that each output voxel's complete result is produced within one unified operation. SpConv v2 [6] pioneered this approach for sparse convolution, introducing bitmask-based sorting to reorder output voxels so that those with similar active-neighbor patterns are grouped together, enabling warp-level skipping of contiguously empty offset positions. TorchSparse++ [17] later showed that with sufficiently optimized GEMM tile configurations, the full implicit GEMM without explicit skipping can match or exceed the performance of sort-and-skip, because the overhead of sorting and the limited skip granularity often offset the savings from skipping. In both systems, the execution is fully structured and GPU-friendly, but the fused offset dimension inevitably includes empty neighbor positions, so a fraction of arithmetic is spent on neighbors that do not exist and contributes nothing to the final result.

2.3 Orthogonal Optimization Directions

Several lines of work improve sparse convolution performance through mechanisms orthogonal to the execution strategy itself. *Compiler-based* optimizations (TorchSparse++ TVM codegen [17], PCEngine [16]) apply operator fusion and hardware-specific code generation within an existing execution model. *Profiling-based autotuning* (e.g., PCEngine [16] switching between gather-scatter and implicit GEMM based on layer density, TorchSparse++ [17] autotuning tile configurations) adapts execution parameters but does not change the underlying execution model. *Hardware acceleration* approaches (e.g., PointAcc [10]) change the compute substrate but not the software-level execution model. These directions are orthogonal to GTSparse: GTSparse proposes a new execution model, and the above optimizations remain applicable on top of it.

2.4 Motivation: The Effective Throughput Gap

In the target inference scenario (batch size 1, single frame per forward pass), sparse convolution layers dominate end-to-end latency. The total time spent on each layer includes both preprocessing (building the neighbor map or runtime metadata) and the convolution kernel itself. Reducing either component directly lowers end-to-end inference time. To understand where existing systems lose efficiency, we distinguish two throughput metrics:

- *Raw computation throughput*: total FLOPs issued by the hardware divided by wall-clock time, regardless of whether the computation is useful. This is what standard GPU profilers report.
- *Effective computation throughput*: only the FLOPs corresponding to genuinely active pairs, divided by wall-clock time.

We formalize effective throughput as

$$T_{\text{eff}} = \frac{N_{\text{pairs}} \times C_{\text{in}} \times C_{\text{out}} \times 2}{t_{\text{wall}}}, \tag{3}$$

where N_{pairs} is the number of active pairs, C_{in} and C_{out} are the channel widths, the factor of 2 accounts for multiply and accumulate, and t_{wall} is the total wall-clock time of the sparse convolution layer including both preprocessing and kernel execution. A system that wastes arithmetic on empty neighbors will show raw throughput much higher than effective throughput; a system that spends excessive time on preprocessing will show low effective throughput even if its kernel is efficient. Only when both proportionality and speed are high does T_{eff} approach the GPU's peak.

Figure 1 illustrates this on a representative SubM 3×3×3 layer from the SECOND [19] backbone running on KITTI [7] inputs, measured on an NVIDIA RTX 3080. MinkowskiEngine performs almost exclusively useful arithmetic (raw ≈ effective), but both values are low because per-offset fragmentation prevents the GPU from reaching high throughput.

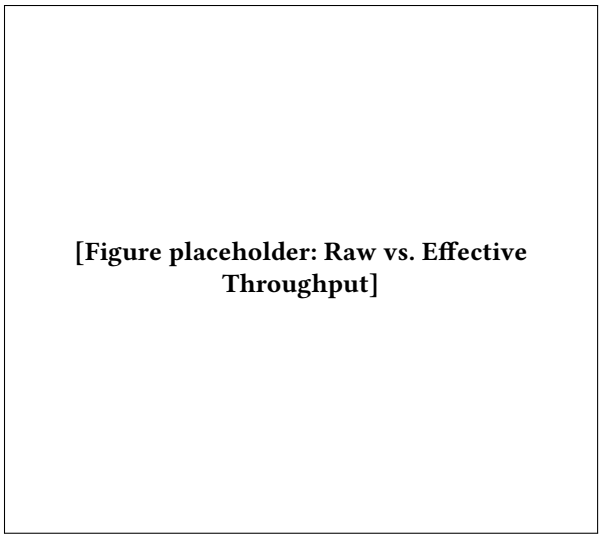


Figure 1. Raw vs. effective computation throughput for existing sparse convolution engines on a representative SubM layer from SECOND on KITTI. MinkowskiEngine wastes little arithmetic but achieves low absolute throughput due to fragmentation. TorchSparse++ achieves the highest raw throughput but only ~XX% translates to effective throughput. SpConv v2 sits in between: its sort-and-skip mechanism reduces wasted computation relative to TorchSparse++, but preprocessing overhead lowers overall throughput.

TorchSparse++ achieves the highest raw throughput by executing a fully structured GEMM, but only about XX% of that computation corresponds to active pairs; the rest is wasted on empty neighbors. SpConv v2 reduces wasted computation through sort-and-skip, narrowing the gap between raw and effective throughput, but its preprocessing overhead (sorting, hash-table construction) lowers the overall wall-clock throughput below TorchSparse++.

No existing system achieves both high raw throughput and high proportionality simultaneously, leaving substantial room for improvement.

3 Overview

Figure 2 shows the end-to-end workflow of GTSparse for a single sparse convolution layer. The system operates in three steps: *template classification* determines which geometric template each output voxel belongs to, *layout construction* arranges all voxels into a contiguous tile stream segmented by template, and *kernel execution* processes the entire stream in a single GPU launch. We describe each step below.

Template classification. Given the active input voxel coordinates, the runtime builder probes, for each output voxel, which of its potential neighbors are present, producing a per-voxel bitmask of active offsets. The builder then classifies each output voxel to the narrowest geometric template

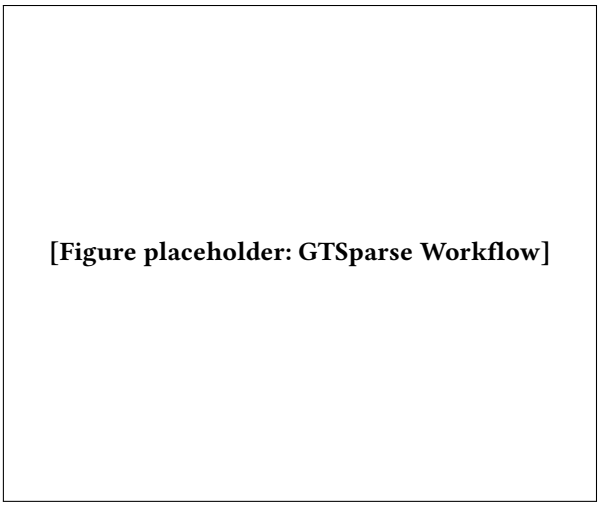


Figure 2. GTSparse workflow for a single sparse convolution layer. Given input voxel coordinates, the runtime builder classifies each output voxel to a geometric template, arranges all voxels into a contiguous tile stream segmented by template, and executes the entire stream in a single GPU kernel where each tile performs a dense matrix multiplication over only its template's offsets.

whose offset subset covers this bitmask, testing templates from narrowest to widest and a full-width fallback guarantees that every output voxel is assigned. Crucially, this requires no global sorting and no device-to-host synchronization, as each voxel is classified independently in parallel on the GPU.

Runtime layout construction. After classification, output voxels are grouped by their assigned template and arranged into a single contiguous stream. Each group is padded to tile boundaries so that every tile contains only voxels of the same template. The kernel reads the template identity once per tile to determine its compute path, with no further scheduling decisions needed at kernel time.

Kernel execution. A single GPU kernel launch processes the entire tile stream. Each tile reads its template identity and executes a dense matrix multiplication over only that template's offsets, so arithmetic is spent only on offsets known to contain active neighbors. In all cases, the computation within a tile is a uniform, Tensor Core compatible dense GEMM, preserving the structured execution that makes implicit-GEMM engines fast. Because tiles from different templates coexist in the same launch, the GPU processes the heterogeneous workload without multiple dispatches or cross-template synchronization. This resolves the *heterogeneous execution* challenge identified in §1. The remaining *template set design* challenge (how to select templates that achieve tight coverage) will be elaborated in §4.

Together, these three steps give GTSparse a complete execution path that is both lightweight and efficient. The overall pipeline preserves the high raw throughput of implicit-GEMM engines while approaching the proportionality of per-offset engines, directly improving effective computation throughput as defined in §2.4.

4 Design

This section details the design of GTSparse. We first describe the offline template set construction (§4.1), which produces the fixed geometric template set used across all inputs. We then describe the runtime components that correspond to the pipeline in §3: the runtime builder that performs template classification and layout construction (§4.2), and the kernel execution model that processes the resulting tile stream (§4.3).

4.1 Geometric Template Set Construction

The template set design challenge, as identified in §1, is to select a compact set of geometric templates such that, after assigning each output voxel to its narrowest compatible template, the average assigned width is significantly smaller than the full offset count. We frame this as a structured variant of set cover and solve it with a two-stage pipeline: geometry-driven candidate generation followed by data-driven greedy selection.

Problem formulation. Recall from §2.1 that a sparse convolution kernel defines K spatial offsets $\mathcal{D}$, and each output voxel $p \in \mathcal{A}_{\text{out}}$ has a set of valid offsets $\mathcal{N}(p) \subseteq \mathcal{D}$. A *template set* is a collection of offset subsets $\mathcal{T} = \{T_1, \ldots, T_m\}$, $T_i \subseteq \mathcal{D}$. Each output voxel p is assigned to its narrowest compatible template, i.e., the smallest T_i that contains all of p's active offsets:

$$a(p) = \arg\min_{T_i \in \mathcal{T}} |T_i| \quad \text{s.t.} \quad \mathcal{N}(p) \subseteq T_i. \tag{4}$$

The objective is to minimize the average assigned width:

$$\min_{\mathcal{T}} \frac{1}{|\mathcal{A}_{\text{out}}|} \sum_{p \in \mathcal{A}_{\text{out}}} |a(p)| \quad \text{s.t.} \quad \mathcal{T}_0 \subseteq \mathcal{T}, \tag{5}$$

where $\mathcal{T}_0$ is a fixed *base template set* (defined below) that must always be included.

Base templates. Any template set includes two base templates by default:

- **Center** (width 1): covers output voxels whose only active neighbor is the center offset. These voxels are common at very low densities.
- **Full-width fallback** (width K): covers all offsets. It works as a fallback template guaranteeing that every output voxel can be assigned regardless of its active pattern.

Candidate family generation. Rather than searching the exponential space of all possible offset subsets (2^K possibilities), we exploit the geometric structure of the kernel to define a tractable candidate space. We first fix a canonical ordering of the K offsets into a sequence. On this sequence, we define three parametric families of candidate templates:

Let $\sigma = (\sigma_0, \sigma_1, \ldots, \sigma_{K-1})$ denote the canonical offset sequence. We define three parametric families of candidate templates:

- **Contiguous-hole**(h, s). Retain all offsets except a contiguous segment of length h starting at position s:
$$T = \{\sigma_i \mid (i - s) \bmod K \geq h\} \cup \{\text{center}\}.$$
- **Periodic-skip**(n, q, s). Starting at position s, remove n out of every q consecutive offsets:
$$T = \{\sigma_i \mid (i - s) \bmod q \geq n\} \cup \{\text{center}\}.$$
- **Hybrid**(n, q, h, s). Apply both a periodic-skip pattern and a contiguous hole simultaneously.

In all three types, the center offset is unconditionally included regardless of the parametric pattern. The same template set is used for both submanifold and full convolution layers; under submanifold convolution ($\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$), the center offset is always active by definition, so any template that excludes it cannot cover any output voxel.

For each combination of parameters other than s (e.g., h for contiguous-hole, or n, q for periodic-skip), we fix these parameters and collect all valid instantiations over s into a single *family* F, which serves as the atomic unit of selection. This grouping ensures that when a family is selected, all of its phases (all possible s) are available for assignment; otherwise, voxels whose active patterns align only with a specific phase would lose coverage. The candidate space C is then the set of all families obtained by enumerating all valid parameter combinations across the three family types (contiguous-hole, periodic-skip, hybrid). A voxel p is *covered* by a family if at least one of its variants T satisfies $\mathcal{N}(p) \subseteq T$. Since C is derived entirely from the kernel's offset geometry, it does not depend on any particular input data.

Greedy selection. We solve the optimization problem in Eq. 5 approximately using a greedy algorithm over the candidate space C. The core idea is to iteratively add the family that maximally reduces the objective (average assigned width) at each step.

To quantify the gain of adding a candidate family F to the current template set $\mathcal{T}$, we define its *marginal offset reduction*. Let $a_{\mathcal{T} \cup F}(p)$ denote the assignment of voxel p under the augmented set $\mathcal{T} \cup F$. The marginal offset reduction is:

$$\Delta(F) = \sum_{p \in \mathcal{A}_{\text{out}}} \left(|a(p)| - |a_{\mathcal{T} \cup F}(p)| \right), \tag{6}$$

which measures the total number of offsets eliminated if F were added. Starting from the base set $\mathcal{T}_0$, the algorithm selects the family with the largest Δ, adds it to $\mathcal{T}$, updates

Algorithm 1 Greedy Template Set Selection

Require: Candidate space $\mathcal{C}$, base set $\mathcal{T}_0$, representative output voxels $\mathcal{A}_{\text{out}}$, max rounds R, threshold ϵ
Ensure: Template set $\mathcal{T}$

1: $\mathcal{T} \leftarrow \mathcal{T}_0$
2: Compute initial assignment $a(p)$ for all $p \in \mathcal{A}_{\text{out}}$
3: **for** round $= 1, 2, \ldots, R$ **do**
4: **for** each $F \in \mathcal{C}$ **do**
5: Compute $\Delta(F)$ via Eq. (6)
6: **end for**
7: $F^* \leftarrow \arg\max_F \Delta(F)$
8: **if** $\Delta(F^*) < \epsilon$ **then**
9: **break**
10: **end if**
11: $\mathcal{T} \leftarrow \mathcal{T} \cup F^*$; $\mathcal{C} \leftarrow \mathcal{C} \setminus \{F^*\}$
12: Update $a(p)$ for all $p \in \mathcal{A}_{\text{out}}$
13: **end for**
14: **return** $\mathcal{T}$

all assignments, and repeats until Δ falls below a threshold ϵ.

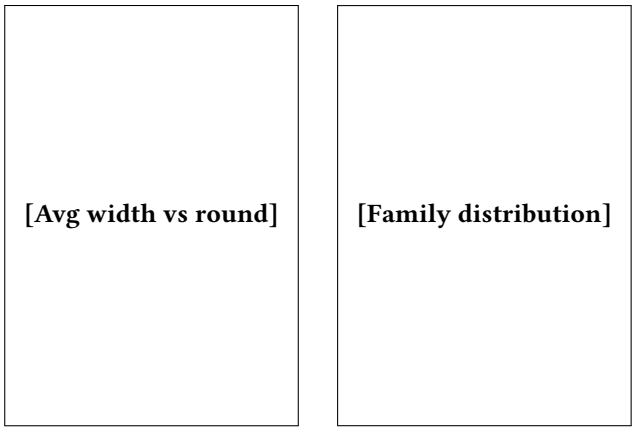
We formalize this as Algorithm 1. Line 1 initializes the template set to the base set. Line 2 computes the initial voxel assignments under $\mathcal{T}_0$. The main loop (lines 3–10) evaluates all remaining candidates, selects the best one, and updates assignments. The algorithm terminates either when the budget m is exhausted or when the marginal gain becomes negligible. The selection operates on a representative set of output voxels sampled from real workloads (e.g., multiple SubM layers from SECOND on KITTI).

Resulting template set. We run Algorithm 1 with $\epsilon = 0.1$, using the first 16 frames of KITTI with SECOND [19] as the representative sample $\mathcal{A}_{\text{out}}$ (aggregating output voxels across all SubM layers). As shown in Figure 3a, the algorithm converges in two rounds for 3x3x3 convolution kernels, selecting:

- Round 1: periodic-skip$(2, 3)$, producing 3 variants of width 10.
- Round 2: periodic-skip$(1, 3)$, producing 3 variants of width 19.

The widths (10 and 19) include the unconditionally added center offset. Together with the base set $\mathcal{T}_0$ (center of width 1, full-width fallback of width K), the final template set contains 8 templates.

Figure 3b shows the template assignment distribution. Approximately 70% of output voxels are assigned to templates narrower than the full-width fallback, reducing the average template width to XX (vs. 27 for full-width fusion). Importantly, the template set is input-independent: constructed once offline from a representative sample, it achieves stable coverage across different datasets (KITTI, nuScenes, SemanticKITTI) without modification.



(a) Average assigned width per greedy round.

(b) Output voxel distribution across families.

Figure 3. Greedy template set selection on KITTI+SECOND. (a) Average assigned template width converges rapidly in two rounds. (b) Approximately 70% of output voxels are assigned to narrower templates (width < 27).

4.2 Runtime Builder

Given the template set $\mathcal{T}$ produced offline, the runtime builder constructs, for each input frame, the execution metadata that the kernel consumes. It implements the template classification and layout construction steps described in §3.

Template classification. For each output voxel p, the builder first determines $\mathcal{N}(p)$ by probing which of the K neighbor positions are occupied. Representing $\mathcal{N}(p)$ as a bitmask $b(p) \in \{0,1\}^K$, we precompute for each template $T_i \in \mathcal{T}$ a *reject mask* $r_i \in \{0,1\}^K$ corresponding to $\mathcal{D} \setminus T_i$ (the offsets not in T_i). The assignment (Eq. 4) is then computed as:

$$a(p) = \arg\min_{T_i \in \mathcal{T}} |T_i| \quad \text{s.t.} \quad b(p) \ \texttt{AND} \ r_i = 0, \tag{7}$$

which is equivalent to Eq. 4 but reduces the subset test $\mathcal{N}(p) \subseteq T_i$ to a single bitwise operation. Each voxel's classification is independent, requiring $O(m)$ bitwise operations per voxel.

Tile stream layout. Let $\mathcal{P}_i = \{p \in \mathcal{A}_{\text{out}} : a(p) = T_i\}$ be the set of voxels assigned to template T_i, and let B_M be the tile size. The builder pads each group to $\hat{n}_i = \lceil |\mathcal{P}_i| / B_M \rceil \cdot B_M$ voxels (filling padding positions with invalid markers), then concatenates all groups into a contiguous tile stream of total length $N = \sum_i \hat{n}_i$. This layout guarantees that every tile of B_M consecutive positions contains only voxels of the same template. The kernel reads the template identity once per tile to determine its compute path, with no further scheduling decisions needed. Padding positions produce zero contributions during computation and are skipped during writeback.

Algorithm 2 Template-Driven Kernel Execution Model

for each tile (b_m, b_n) in the tile stream **in parallel do**
  $T_i \leftarrow$ template of tile b_m
  Initialize accumulator $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
  for $j = 1, \ldots, |T_i|$ **do**
    for $k = 1, \ldots, \lceil C_{\text{in}}/B_K \rceil$ **do**
      $\mathbf{X} \leftarrow$ gather input block from offset d_j, channel tile k
      $\mathbf{W} \leftarrow$ load weight block for offset d_j, channel tile k
      $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{W}$
    end for
  end for
  Write $\mathbf{Y}$ to output (skip padding positions)
end for

Overhead and reuse. The builder executes entirely on the GPU with no device-to-host synchronization. Its time complexity is $O(|\mathcal{A}_{\text{out}}| \cdot (K + m))$: $O(K)$ per voxel to probe neighbors and $O(m)$ for classification. With our template set ($m = 8$, $K = 27$), $K > m$ and thus neighbor probing dominates. For submanifold convolution layers ($\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$), the assignment $a(p)$ and the tile stream are invariant across layers sharing the same kernel size, so the metadata is built once and reused. For full convolution layers, the builder runs once per layer.

4.3 Kernel Execution Model

The kernel computes Eq. 2 for all output voxels by consuming the tile stream. Its execution adopts the same implicit-GEMM tiling strategy as existing engines (SpConv v2, TorchSparse++), with one modification: the reduction dimension (number of offsets) is no longer fixed at K for all tiles, but equals $|a(p)|$ for each tile according to its assigned template.

Tiling structure. The workload is decomposed into tiles of size $B_M \times B_N$ (output voxels × output channels), identical to standard implicit GEMM. The inner loop reduces over offsets and input channels in steps of B_K. In standard implicit GEMM, this loop runs for $K \times \lceil C_{\text{in}}/B_K \rceil$ steps. In GTSparse, it runs for $|a(p)| \times \lceil C_{\text{in}}/B_K \rceil$ steps, where $|a(p)| \leq K$. Each step is identical: gather input features, load the corresponding weight slice, and perform a dense matrix-multiply-accumulate. The only difference is how many steps each tile executes.

Heterogeneous dispatch. A single kernel launch processes the entire tile stream. Algorithm 2 summarizes the execution model.

Because the tile stream layout guarantees that all B_M voxels within a tile share the same template, the template identity is read once per tile. Tiles with narrow templates complete in fewer iterations; tiles with the full-width fallback execute the same number of iterations as standard implicit GEMM. No cross-tile coordination is needed. The execution is intra-template structured (each tile runs a uniform dense GEMM) and inter-template approaching exact work (narrower tiles skip offsets known to be empty), improving T_{eff} (Eq. 3).

References

[1] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jurgen Gall. 2019. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *ICCV*.

[2] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. 2020. nuScenes: A Multimodal Dataset for Autonomous Driving. In *CVPR*. https://doi.org/10.1109/CVPR42600.2020.01164

[3] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. 2023. VoxelNeXt: Fully Sparse VoxelNet for 3D Object Detection and Tracking. In *CVPR*.

[4] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *CVPR*.

[5] Angela Dai, Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Christian Theobalt. 2017. BundleFusion: Real-Time Globally Consistent 3D Reconstruction Using On-the-Fly Surface Reintegration. *ACM Trans. Graph.* 36, 3 (2017). https://doi.org/10.1145/3054739

[6] Lue Fan, Feng Wang, Ziqi Pang, Lei Zhu, Zhicheng Su, Fang Chen, Naiyan Wang, and Zhaoxiang Zhang. 2022. Embracing Single Stride 3D Object Detector with Sparse Transformer. In *CVPR*.

[7] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *CVPR*. https://doi.org/10.1109/CVPR.2012.6248074

[8] Shahram Izadi, David Kim, Otmar Hilliges, David Molyneaux, Richard A. Newcombe, Pushmeet Kohli, Jamie Shotton, Steve Hodges, Dustin Freeman, Andrew J. Davison, and Andrew W. Fitzgibbon. 2011. KinectFusion: Real-time 3D Reconstruction and Interaction Using a Moving Depth Camera. In *UIST*. https://doi.org/10.1145/2047196.2047270

[9] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. PointPillars: Fast Encoders for Object Detection From Point Clouds. In *CVPR*. https://doi.org/10.1109/CVPR.2019.01298

[10] Yujun Lin, Zhekai Wang, Yingyan Ding, and Song Han. 2021. PointAcc: Efficient Point Cloud Accelerator. In *MICRO*.

[11] Zhijian Liu, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela L. Rus, and Song Han. 2023. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation. In *ICRA*. https://doi.org/10.1109/ICRA48891.2023.10160968

[12] Tixiao Shan and Brendan J. Englot. 2018. LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain. In *IROS*. https://doi.org/10.1109/IROS.2018.8594299

[13] Tixiao Shan, Brendan J. Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus. 2020. LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping. In *IROS*. https://doi.org/10.1109/IROS45743.2020.9341176

[14] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, Vijay Vasudevan, Wei Han, Jiquan Ngiam, Hang Zhao, Aleksei Timofeev, Scott Ettinger, Maxim Krivokon, Amy Gao, Aditya Joshi, Yu Zhang, Jonathon Shlens, Zhifeng Chen, and Dragomir Anguelov. 2020. Scalability in Perception for Autonomous Driving: Waymo Open Dataset. In *CVPR*. https://doi.org/10.1109/CVPR42600.2020.00252

[15] Haotian Tang, Zhijian Liu, Shengyu Zhao, Yujun Lin, Ji Lin, Hanrui Wang, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *MLSys*.

[16] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. PCEngine: Efficient Point Cloud Inference Engine. In *MLSys*.

[17] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *MICRO*.

[18] Thomas Whelan, Stefan Leutenegger, Renato F. Salas-Moreno, Ben Glocker, and Andrew J. Davison. 2015. ElasticFusion: Dense SLAM Without A Pose Graph. In *Robotics: Science and Systems*. https://doi.org/10.15607/RSS.2015.XI.001

[19] Yan Yan, Yuxing Mao, and Bo Li. 2018. SECOND: Sparsely Embedded Convolutional Detection. *Sensors*.

[20] Ji Zhang and Sanjiv Singh. 2014. LOAM: Lidar Odometry and Mapping in Real-time. In *Robotics: Science and Systems*. https://doi.org/10.15607/RSS.2014.X.007